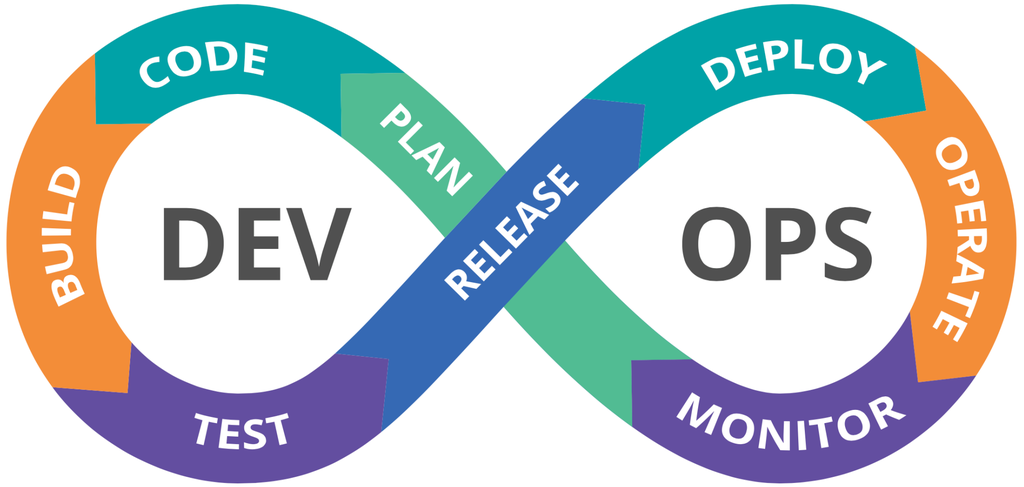
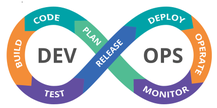


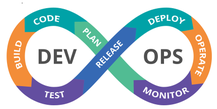
Tehničke prakse za uspostavljanje povratnih tokova
Povratne informacije za povećanje kvaliteta putem revizije koda

predavanja su bazirana i slike preuzete iz knjige
*“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”*
Gene Kim, Jez Humble, Patrick Debois, John Willis



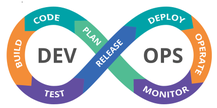
Tehničke prakse za uspostavljanje povratnih tokova

- Tehničke prakse koje omogućavaju uspostavljanje kvalitetnih povratnih informacija podrazumevaju:
  - kreiranje odgovarajuće telemetrije koja omogućava da se problemi uoče i reše
  - korišćenje telemetrijskih podataka za bolje predviđanje ponašanja sistema i dostizanja ciljeva
  - obezbeđivanje povratnih informacija koje omogućavaju razvojnim i operativnim timovima da bezbedno isporučuju i instaliraju softver
  - integrisanje korisničkih iskustava i povratnih informacija u rad razvojnih timova
  - **obezbeđivanje povratnih informacija za povećanje kvaliteta putem revizije koda od strane kolega i programiranja u parovima**



Značaj revizije koda

- Cilj je smanjiti rizik da implementacija izmena u kodu dovedu do problema u produkciji.
- Često se u tradicionalnim metodama oslanjamo na posebne timove koji bi radili reviziju koda, ali ti timovi mogu biti nesvesni konteksta u kome će se kod koristiti i sam proces revizije može trajati dugo
- Jedan od primera uspešnog uvođenja revizije koda u praksu je GitHub-ov koncept *pull request-a.*



Značaj revizije koda

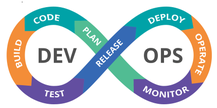
- Cilj je smanjiti rizik da izmene u kodu dovedu do problema u produkciji.

- Često se u tradicionalnim metodama oslanjamo na posebne timove koji bi radili reviziju koda, ali ti timovi mogu biti nesvesni konteksta u kome će se kod koristiti i sam proces revizije može trajati dugo.

- Jedan od primera uspešnog uvođenja u praksu revizije koda od strane bliskih saradnika je GitHub-ov koncept *pull request-a.*

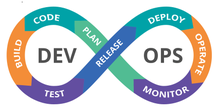
  - Ovaj mehanizam zapravo omogućava da programer obavesti neposredno okruženje o promenama koje je postavio na sistem za verzioniranje i da zatraži da neko od kolega proveri kod.

    - Ljudi koji rade na istom projektu mogu lako proceniti da li će taj kod imati neke negativne efekte na ostatak koda



GitHub pull request flow

- Pet koraka
  - za rad na nekoj funkcionalnosti programer kreira posebnu granu izdvojenu iz glavne razvojne grane
  - tokom razvoja izmene se komituju na tu lokalnu granu
  - kada smatraju da je razvoj gotov u meri da može da se spoji sa glavnom granom, postavljaju *pull request*
  - kolege pregledaju, diskutuju i odobravaju pull request (može se specificirati +1, +2 i sl). Kada se prikupi traženi broj odobrenja radi se spajanje na glavnu granu
  - nakon spajanja se glavna grana sa pridodatim novim kodom propušta kroz deployment pipeline
- Ovakav postupak omogućava da se isporuči veliki broj *releasa,* sa malim rizikom grešaka



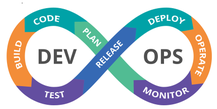
Opasnosti koje nosi klasični proces odobravanja

- Katastrofalni otkaz koji se desio nakon 15-minutnog deploymenta sa greškom doveo je do 440 miliona $ gubitaka i uzrokovao da kompanija The Knight Capital bude prodana odmah nakon incidenta kako bi se sprečilo dalje širenje gubitaka

- Dve narativa kako je do greške došlo (ali nisu baš činjenično tačna)

  - problem sa upravljanjem *change managementom*

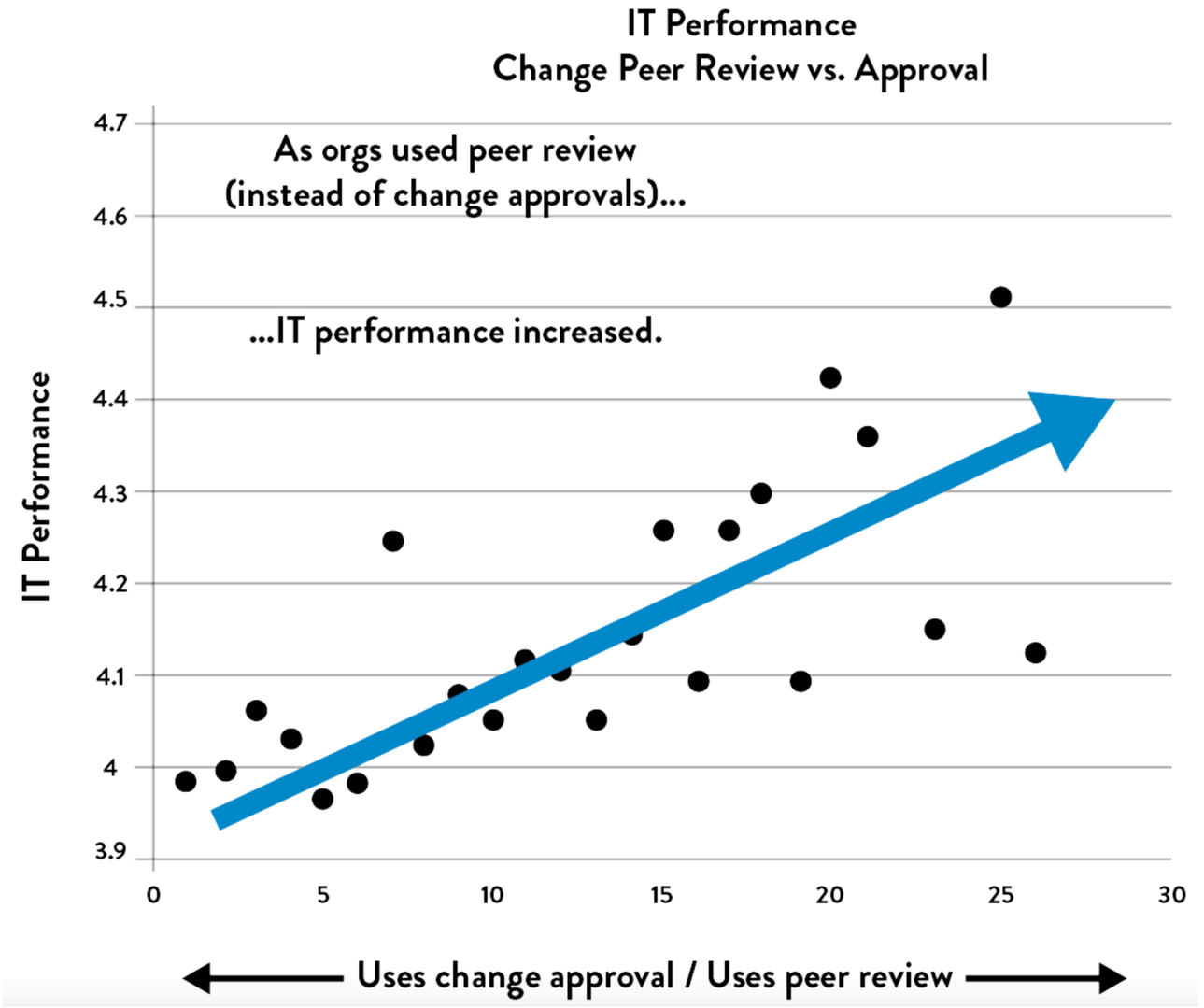
  - greška počinjena tokom testiranja

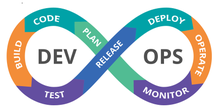


Opasnosti koje donosi prevelika kontrola procesa implementacije izmena

- Tradicionalni proces odobravanja izmena može dovesti do problema sa predugim vremenima isporuke i usporavanjem protoka povratnih informacija iz procesa isporuke
- Kada se greška desi reakcija je često
  - dodavanje novih pitanja u formu zahteva za implementaciju izmene
  - zahtevanje dodatnih odobrenja
  - traženje da se isporuka odloži kako bi se zahtev za izmenu dodatno evaluirao
- Sve ovo može zapravo dovesti do više tenzija u samom procesu deploymenta jer se povećava broj koraka, čekanjem se povećava količina koda koja će odjednom biti isporučena, odobravanje sprovode ljudi koji su udaljeni od samog procesa razvoja
- Pokazuje se da klasični proces odobravanja implementacije vodi do drastično lošijih rezultata u odnosu na proces odobravanja od strane timskih kolega (*peer review*)
- IT kompanije koje koriste *peer review* postižu znatno bolje performanse

Opasnosti koje donosi prevelika kontrola procesa implementacije izmena



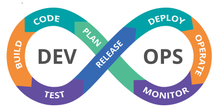


Omogućavanje koordinacije i planiranje implementacije izmena

- Koordinacija je neophodna kada god timovi rade na sistemima koji imaju zajedničke zavisnosti

- Što je naš sistem manje čvrsto spregnut, manja je potreba za bliskom koordinacijom između različitih razvojnih timova, kada je arhitektura zaista servisna onda timovi imaju slobodu odlučivanja

- Ipak, čak i u slabo spregnutim kompleksnim arhitekturama postoji rizik da će izmene koje u sistem unese jedan tim imati negativan efekat na funkcionalnosti koje razvija drugi tim
  - neophodno je obezbediti kanale kojim timovi unapred mogu najaviti promene koje će uneti u sistem (chat-rooms, Slack…)
  - u vrlo kompleksnim sistemima i sistemima u kojima postoji veći stepene međuzavisnosti dobro je planirati terminski implementaciju izmena, gde predstavnici svih timova zajedno donose odluku o planu i redosledu implementacije kako bi se minimizovao rizik pojave problema u produkcionim okruženjima



Omogućavanje revizije od strane kolega (peer review)

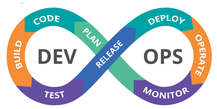
- Umesto da postoji eksterno telo koje odobrava izmene možemo se osloniti na kolege iz tima da obave reviziju i odobre izmene

- Code review se traži pre spajanja koda na glavnu razvojnu granu na kojoj će efekat promena biti vidljiv i potencijalno problematičan za ceo tim

  - osim revizije od strane kolega, za kritične stvari može se tražiti još jedna revizija od strane osobe u timu koja je specijalizovana za određenu tehnologiju

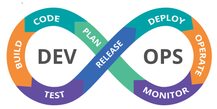
- I ovde važi princip malih paketa koda - što više koda treba odjednom pregledati to je manja verovatnoća da će onaj ko radi reviziju uočiti potencijalne probleme

- Preporuke za reviziju:
  - svako treba da ima bliskog saradnika, kolegu koji može da pregleda kod
  - svako treba da nadgleda tok komitova i prijavi potencijalne konflikte
  - definisati koje izmene su posebno rizične i zahtevati dodatnu proveru
  - ako se napravi izmena koja je previše velika, a da nije jasno zašto bi bilo neophodno odjednom primeniti ceo veliki paket izmena, razložiti izmene na manje pakete



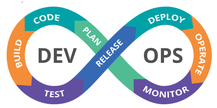
Omogućavanje revizije od strane kolega (peer review)

- Revizija koda se može obaviti na nekoliko načina
  - Programiranje u paru
  - “preko ramena”
  - Prosleđivanje notifikacije za reviziju emailom
  - Revizija potpomognuta alatima (GitHub pull request, CodeCommit pull requet, Mercurial, Subversion…)



Opasnosti od dodatnih manuelnih testova i zamrzavanja koda

- Kada se desi da zbog greške ili propusta u testovima dođe do vidljive greške na produkciji prirodna reakcija je da se poveća broj testova
  - ali ako se samo povećava broj testova koji se izvode u završnoj fazi, ovo ne mora dovesti do pozitivnih rezultata, pogotovo u slučaju manuelnih testova
    - manuelni testovi traju dugo, odlažu deployment, samim tim se gomila količina koda koju treba implementirati odjednom, što sve dovodi do povećanja rizika da će se ponovo desiti greška
- Automatizovani testovi, potpuno integrisani u deployment pipeline otklanjaju ovaj problem - ako je propust bio u testiranju, rešenje pokušati pokriti uočeni problem automatizovanim testovima koji će se izvršavati u svakom ciklusu deployment pipeline-a



Pair programming kao tehnika za unapređenje isporuke koda

- Tehnika predložena u Extreme i Agile pristupu razvoju - dva programera rade zajedno na istoj radnoj stanici

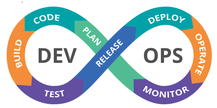
- Jedan programer (*driver*) je u ulozi onoga ko aktivno menja kod, a drugi (*navigator, observer, pointer*) je prisutan i nadgleda izmene kako one nastaju.

  - Tokom ovog procesa *observer* može predložiti izmene predviđajući buduće probleme koje trenutni način implementacije može da donese.

  - Na ovaj način *driver* se fokusira na samu implementaciju

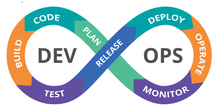
- Drugi obrazac primene programiranja u paru je kod TDD, gde jedan progrmaer unapred piše potrebne testove, a drugi piše kod koji treba da zadovolji postavljene testove.

- Dodatna korist od programiranja u paru je što se znanje o određenim delovima koda širi kroz tim - više od jedne osobe je upoznato sa tim šta kod i kako radi



Analiza efikasnosti pull procesa

- Kako je revizija koda od strane saradnika važan deo kontrolisane isporuke koda, neophodno je imati i neku informaciju da li taj proces radi na zadovoljavajući način.
  - jedan metod je da se u slučaju otkrivanja nekih problema analizira kako je izvršena revizija koda koji je izazvao grešku i da se ona unapredi
- Dobar i loš pull request ne moraju da se procenjuju na osnovu rezultata same implementacije, već na osnovu konteksta, dobar pull request sadrži sve relevantne podatke šta i zašto je menjano, koji je cilj izmene i od kog zahteva ta izmena potiče.



Treba stalno uklanjati birokratske procese

- Mnoge kompanije i dalje imaju ustanovljene procedure odobravanja implementacije izmena koje mogu potrajati i mesecima
  - one suštinski “ubijaju” agilnu isporuku, a istovremeno (iz već opisanih razloga) potencijalno uvećavaju mogućnost pojave problema u produkciji
- Dobra metrika da se proceni koliko je proces zakomplikovan je ukupan broj tiketa je neophodan da bi se prošao ceo proces implementacije izmena
  - ključni cilj je redukovati što više trud koji programeri moraju da ulažu u ovaj proces
- Neophodno je stalno raditi reviziju ovog procesa i uklanjati sve procedure alate i odobrenja koja blokiraju redovne isporuke koda